

03 - Accounts and Financial State

Feature objective

Create a trustworthy account layer that lets the product reason about balances without pretending SMS-derived estimates are live bank balances.

Accounts are part of the financial model but are not a primary tab.

Dependencies

- Foundation/App Shell
- Auth
- backend accounts/financial-state endpoints

In scope

- account list
- add/edit account
- account detail
- manual balance update
- balance provenance
- freshness status
- current vs estimated/confirmed balance
- account-linked transaction preview
- financial-state summary hooks used by Home/Plan

Out of scope

- live bank connection
- Account Aggregator
- brokerage accounts with execution
- statement PDF import

Account types

MVP can support:

BANK_SAVINGS
BANK_CURRENT
CASH
CREDIT_CARD
INVESTMENT
LOAN_LINKED if backend models it as an account
OTHER

Keep UI labels human-friendly.

Balance semantics

Never display one generic **balance** without provenance.

Three conceptual levels:

1. Live/authoritative - future connected source
2. Confirmed - manual confirmation, trusted SMS balance checkpoint, statement/API source
3. Estimated - calculated from a confirmed point plus tracked transactions

UI examples:

INR 18,331
Confirmed today via bank SMS

or:

INR 18,620 estimated
Based on tracked transactions

Freshness policy

fresh: under 7 days
stale: 7-30 days
very stale: over 30 days

Daily dashboard may use estimates.

Before serious plan regeneration:

- stale major balances -> warning
- very stale major balances -> strong confirmation prompt
- user can still continue unless confidence is unusably low

Routes

```
accounts/  
|-- index.tsx  
|-- new.tsx  
|-- [id].tsx  
`-- [id]/edit.tsx
```

Accounts are reachable from Profile/Financial Details and contextual links from Home/Plan.

Feature folder

```
features/accounts/  
|-- screens/  
|-- components/  
|   |-- AccountRow/  
|   |-- BalanceStatus/  
|   |-- FreshnessBadge/  
|   `-- BalanceUpdateSheet/  
|-- hooks/  
|-- services/  
|-- forms/  
|-- schemas/  
|-- utils/  
`-- types.ts
```

Accounts list screen

For each account show:

- account name
- masked identifier when available
- balance
- confirmed vs estimated
- last updated
- source

Example:

SBI Savings -3812
INR 18,331
Confirmed 2h ago - SMS

Add Account is available but not over-emphasized.

Add Account

Required:

- account type
- display name
- current balance

Optional:

- institution
- last four digits
- balance as-of date

If adding credit card:

- credit limit optional
- statement date optional
- payment due date optional

Account detail

Sections:

1. balance and provenance
2. freshness/status
3. update balance action
4. recent transactions
5. monthly inflow/outflow summary
6. linked goals when relevant
7. balance history if backend provides it

Manual balance update

When the user updates balance, treat it as a confirmation/checkpoint, not as a fake transaction unless backend accounting explicitly requires a reconciliation adjustment.

Backend should preserve previous values/history.

Financial-state aggregate

Expose a feature hook consumed by Home/Plan:

`useFinancialSnapshot()`

It should return server-computed/normalized data such as:

- liquid cash estimate
- confirmed cash amount
- total tracked assets
- liabilities
- monthly income
- monthly spending
- surplus
- stale critical inputs

Mobile does not independently recompute canonical financial state from UI records.

API requirements

Conceptual:

GET /accounts
POST /accounts

```
GET    /accounts/:id
PATCH /accounts/:id
POST   /accounts/:id/balance-confirmations
GET    /financial-state/current
```

Query hooks

```
useAccounts
useAccount
useCreateAccount
useUpdateAccount
useConfirmAccountBalance
useFinancialSnapshot
```

Invalidate targeted account + financial snapshot queries after balance edits.

Offline behavior

Read:

- cached account list/details may render offline with last-updated label

Write:

- manual balance confirmation should normally require internet in MVP because reconciliation affects canonical financial state
- if later queued offline, it must be an explicit outbox operation with idempotency

Step-by-step implementation

1. Define account UI model using SDK types.
2. Build AccountRow + provenance/freshness UI.
3. Build Accounts list.
4. Build Add Account form.
5. Build Account detail.
6. Build manual balance confirmation flow.
7. Integrate financial snapshot query.
8. Add stale-data warning component reusable by Plan.
9. Add tests for status/freshness rendering.

Acceptance criteria

- user can create, edit, view, and manually confirm an account balance
- UI clearly distinguishes estimated vs confirmed
- stale/very stale state is visible in text, not color only
- no screen labels an SMS-derived estimate as live bank balance
- Home/Plan can consume one financial snapshot contract rather than reconstructing balances independently

Tests

- freshness classification boundary tests
- provenance label rendering
- manual balance form validation
- account query invalidation
- cached offline account render